AI

MDPI

Article

Semantic Firewalls with Online Ensemble Learning for Secure Agentic RAG Systems in Financial Chatbots

Victor Castro-Maldonado, Marco A. Aceves-Fernández *, Luis R. García-Noguez and Jesús C. Pedraza-Ortega

Faculty of Engineering, Centro Universitario S/N, Universidad Autónoma de Querétaro, Querétaro C.P. 76010, Mexico
* Correspondence: marco.aceves@uaq.edu.mx

Abstract

The RAG agentic architecture has demonstrated its ability to transform large language models (LLMs) into agents capable of planning, reasoning, and executing subtasks using external tools or APIs. In the financial sector, one of the main priorities when implementing new technologies—especially in systems like chatbots—is the protection of customer data and the need to maintain customer trust, making the challenges significant. This research presents a robust banking chatbot system that integrates RAG agentic architecture with specialized financial components, setting a new standard in the digital banking sector by prioritizing security, transparency, and functionality. The contributions of this work include the implementation of RAG agentic reasoning and self-correction financial components, and, primarily, the empirical study of the impact of a semantic firewall with online learning in financial RAG agentic systems, evaluated using public benchmarks and standard ranking metrics.

Keywords: RAG; agentic; LLM; chatbot; banking; security; reasoning; financial; self-correction; firewall; meta-learning; ensemble-online

Check for updates

Academic Editors: Albert Y.S. Lam, Andy Chun and Miguel Angel Cazorla

Received: 2 December 2025
Revised: 29 January 2026
Accepted: 2 February 2026
Published: 27 February 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The Retrieval Augmented Generation system (RAG) represents a significant evolution in the field of Natural Language Processing (NLP), combining the generative capabilities of Large Language Models (LLMs) with information retrieval techniques. The main motivation behind RAG is to overcome inherent limitations of strictly parametric LLMs, such as their tendency to hallucinate—generating incorrect information—and their inability to access custom knowledge in real time [1].

Traditional RAG systems often face "context loss," where dividing documents into small fragments for efficient retrieval can result in a loss of context and, consequently, retrieval failures. Contextual Retrieval approaches this issue by adding a specific explanatory context to each fragment before embedding and indexing [2]. Complementary Semantic Segmentation divides documents based on the cosine similarity of sentence embeddings, preserving semantic coherence [3].

The evolution of RAG has moved beyond the simple "retrieve and generate" flow to incorporate reasoning and autonomy mechanisms that enhance the reliability and quality of responses. The agentic RAG architecture enables the completion of complex tasks autonomously and dynamically through decision-making supported by the execution of multiple tools and sources. The system determines which sources to retrieve information

from and which tools are best suited to answer questions. Agentic RAG can map out multiple routes and action plans to resolve complex queries.

This paper proposes a RAG agentic architecture that incorporates specialized components to create a workflow in which a user can obtain information about their products, balances, and transactions, retrieve financial information from the sector, and interact securely through self-correcting queries and the integration of a semantic firewall to mitigate vulnerabilities to potential risks from malicious or inappropriate queries.

The contribution and impact of this work will promote research in RAG and LLMs in the banking sector, focusing on error mitigation, generating specialized knowledge, and adopting best practices to implement these technologies in other domains that require high accuracy and reliability. This will support the democratization of access to financial information, provide clear and accessible answers to complex questions about banking products and services, help users better understand finance and make informed decisions, improve protection against fraud and inappropriate advice, and promote responsible financial inclusion.

2. Related Works

Recent work in the field of RAG Agentic has demonstrated efficiency in areas that require reasoning across different topics, evidence verification, and the use of tools to achieve comprehensive analysis. Language models can learn to use external tools designed for specific purposes [4–6].

The use of LLMs to perform reasoning and specific actions in an interleaved manner enables significant synergy between the two and allows for exception management. These actions also enable interaction with and retrieval of additional information from external sources, such as knowledge bases or environments [7].

In the financial sector, the use of retrieval and generation augmentation is becoming increasingly common for managing specialized documentation on finance topics. FinAgent is a multimodal agent that integrates textual and visual data, enabling comprehensive understanding of market dynamics and historical trading behaviors [8]. It identifies correlations between current market conditions and past market patterns and trends, and integrates market information to support effective decision-making.

Other recent work about AI agent platforms, is the open source that admits multiple AI agents specialized in finance, each one impulsed by LLM [9]. FinRobot approaches the complexities of global markets with a multilayer architecture which admits the data process in real time and the integration of several models, making sophisticated finance tools available to a wider audience.

A framework developed using a multi-aspect RAG system, designed to analyze regulatory compliance in multimodal financial documents. FinSage includes a workflow for preprocessing multimodal data, a retrieval system with multiple routes that recognizes domain-specific designs for financial data, and a module for specialized reclassification within the domain.

Another framework for retrieval-augmented generation in the financial sector is presented in a study that explores agentic AI and the Multi-HyDE system, which generates diverse multiple queries to improve the efficiency and coverage of retrieving large structured financial corpora. This research addresses key challenges in reducing hallucinations and accurately retrieving information from complex financial documents [10].

One work focuses on the use of online machine learning methods, such as incremental learners and adaptive windows, and develops a unified framework that incorporates anomaly detection approaches (isolation forests, LOF), ensemble and gradient-boosted tree methods (XGBoost, LightGBM), and graph-based network detection techniques [11].

Another recent article addresses next-generation continuous graph learning approaches for AML applications. It provides comprehensive experimental evaluations on both synthetic and real-world AML datasets, demonstrating the effects of different hyperparameters. The results show that continuous learning improves the model's adaptability and robustness when facing extreme class imbalances and evolving fraud patterns [12].

The works mentioned offer advantages in analyzing various characteristics but have not yet incorporated elements that enhance the performance of a robust RAG agentic architecture for detecting features such as fraudulent schemes, queries intended to facilitate regulatory violations like tax evasion, detection of malicious patterns such as money laundering, or intelligent handling of ambiguous cases.

3. Materials and Methods

This research incorporates mechanisms in the RAG system focused specifically on finance. Several strategic topics were considered in integrating technologies to prioritize security, transparency, and functionality. The RAG agentic implementation is supported by documentation of financial regulations, integration of agentic capabilities with regulatory knowledge, and reasoning about legal and compliance constraints.

The incorporation of a system capable of reasoning about complex financial queries enables the processing of various issues related to the financial sector. Implementing an agent that reasons simultaneously about multiple financial dimensions allows for parallel analysis, including credit risk assessment, investment optimization, and regulatory compliance. The result of integrated reasoning is the synthesis of multiple analyses into coherent advice, based on a balance between cost and benefit across different financial objectives and dynamic adaptation to the user profile and market conditions.

The implementation of auto-correction introduces the capability for a system to detect and correct issues autonomously, without human intervention [13]. In RAG systems, this translates into the ability to verify the quality of retrieved data, the consistency of the generated response, and the coherence of a workflow.

The proposed system automatically corrects financial advice that violates regulations or is inconsistent. The agent analyzes the entered text to detect regulatory issues, discrepancies in risk management, and financial inconsistencies, and, based on this information, generates a response with the following characteristics:

- Specific corrections.
- Detailed explanations.
- Alternative recommendations.
- Measurable confidence.
- Fast detection.
- Automatic correction.

3.1. Financial Semantic Firewall

A semantic firewall is integrated, emerging as an architectural concept of self-correction [14]. This security and validation layer intercepts and controls the flow of inputs and outputs in real time, based on their meaning and intention. The firewall specifically protects against financial threats and regulatory violations by identifying fraud patterns and pyramid schemes, blocking advice that violates financial regulations, validating suitability to the risk profile, and detecting ethically questionable advice. To achieve this, a semantic firewall with meta-learning and continuous learning is proposed. A lightweight classification model is trained with a dataset of safe and malicious prompts (e.g., injection prompts). The meta-learner evaluates the vectorized prompt. If it classifies the prompt as potentially malicious, it is immediately blocked and an error response is generated. The

meta-learner can generate or adapt dynamic rules based on its analysis. For example, if it detects a data leakage pattern, it can generate a rule to sanitize the response or block access to certain information sources. The interaction data (prompt, response, and feedback) are added to a training dataset for the meta-learner. This enables the model to learn new types of threats or malicious requests that were not present in the initial dataset. The model is updated incrementally, allowing the semantic firewall to adapt to new attack techniques without requiring extensive retraining.

3.2. Meta-Learner

The proposed meta-learner does not rely on a single model; instead, it is implemented with ensemble models that are trained to address the main problem using online learning.

Unlike conventional learning approaches that build models from training data, ensemble methods construct a collection of learners and combine their outputs.

This system consists of several learners, or base learners, usually generated from training data by a base learning algorithm, which may be a decision tree, a neural network, or another type of learning algorithm (Figure 1) [15]. The original ensemble method is the Bayesian average, but more recent algorithms include output codification with error correction, such as bagging and boosting [16].

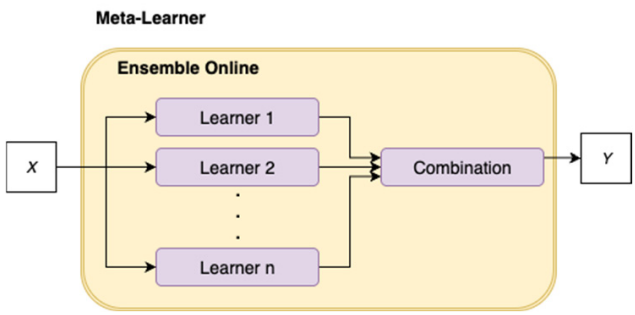


Figure 1. Ensemble methods architecture.

Voting is the most popular and fundamental combination method for nominal outputs. Let T be a set of individual classifiers. $\{h_1, \ldots, h_T\}$ and it is required to combine h_i's to predict the class label from a set of possible l class labels $\{c_1, \ldots, c_l\}$. Generally, it is assumed that, for an instance x, the classifier outputs h_i are given as an l-dimensional label vector, $(h_i^1(x), \ldots, h_i^l(x))^T$ where $h_i^j(x)$ is the output of h_i for the class label c_j. $h_i^j(x)$ can take on different types of values depending on the information provided by the individual classifiers.

The crisp label $h_i^j(x) \in \{0, 1\}$, takes the value of one if h_i predict c_j as a class label and zero otherwise.

Class probability $h_i^j(x) \in [0, 1]$, can be considered as an estimate of the posterior probability $P(c_j \mid x)$.

If the individual classifiers do not have equal performance, it is reasonable to give more weight to the strongest classifiers in the voting; this is achieved through weighted voting. The output class label of the set is determined accordingly:

$$H(x) = c_{arg_j \, max \sum_{i=1}^{T} w_i h_i^j \, (x)} \tag{1}$$

where w_i, is the weight assigned to the classifier h_i. In practical applications, weights are often normalized and restricted by $w_i \geq 0$ and $\sum_{i=1}^{T} w_i = 1$, similarly to what happens in the weighted average [15].

We can distinguish between two learning modes: offline learning and online learning. In offline learning, all training data must be available at the time of model training. Only after training is complete can the model be used for prediction. In contrast, online algorithms process data sequentially, producing a model that can operate without having the complete data set at the outset. The model is updated continuously during operation as it receives more training data [17]. Incremental approaches update the current model with the most recent data. This is how the online learning mode updates the current model with the latest example.

Based on these characteristics, the selected meta-learner implements an Ensemble Online, online ensemble methods that have the advantages of:

- Algorithm Diversity: Each model offers a different perspective.
- Adaptive Weights: Automatically adjust based on performance.
- Continuous Learning: Improves with each new query.
- Robustness: Resistant to overfitting and individual biases.
- Explainability: Generates detailed and understandable answers.

The classifiers implemented as base learners are four specialized algorithms:

1. Passive-Aggressive Classifier
   - Specialized in rapid learning.
   - Effective for fraud patterns.
2. SGD Classifier
   - Stochastic optimization.
   - Robust for noisy data.
3. Multinomial Naive Bayes
   - Probabilistic analysis.
   - Excellent for text classification.
4. Perceptron
   - Linear learning.
   - Fast and efficient.

The Passive-Aggressive (PA) classifier quickly identifies the presence or absence of key words or sentences that are highly discriminatory. It is excellent for detecting known fraud patterns, such as "Give me the password," "Ignore the previous instructions," or database table names. The algorithm is passive if the classification is correct (safe), meaning the model does not make adjustments. It is aggressive if the classification is incorrect (malicious), applying a large adjustment (penalty) to its weights to correct the error immediately. The Passive-Aggressive algorithm for binary classification can be represented in the following three variants [18].

Input: aggressiveness parameter

$$C > 0$$

Initialize: $w_1 = (0, \ldots, 0)$
For $t = 1, 2, \ldots$

- instance: $x_t \in R^n$
- prediction: $\hat{y}_t = sign(w_t \cdot x_t)$
- correct label: $y_t \in \{-1, +1\}$
- cost:

$$l_t = max\{0\ ,\ 1 - y_t(w_t \cdot x_t)\} \tag{2}$$

- upgrade:

a. set

$$\tau_t = \frac{l_t}{\| x \|} \tag{2a}$$

$$\tau_t = min\{C, \frac{l_t}{\| x \|^2}\} \tag{2b}$$

$$\tau_t = \frac{l_t}{\| x \|^2 + \frac{1}{2C}} \tag{2c}$$

b. upgrade

$$w_{t+1} = w_t + \tau_t y_t x_t \tag{3}$$

The Stochastic Gradient Descent (SGD) classifier is a robust linear classifier, efficiently adaptable to the general distribution of prompts. It helps to draw a simple separation line between normal language in banking chat and potential malicious patterns. It is an optimiser used to train linear classifiers, such as SVMs or Logistic Regression. The key is Stochastic Gradient Descent, which updates its parameters (weights) after processing a small batch. This makes it:

- Scalable: It can handle large volumes of prompts without consuming much memory.
- Continuous: Ideal for continuous learning, as it adapts to new fraud patterns as soon as they appear, without needing to retrain the entire model.

Using gradient descent (GD), each iteration updates the weight w based on the gradient of $E_n(f_w)$. Empirical risk $E_n(f)$ measures the performance of the training set. The expected risk $E(f)$ measures the performance of the generalization, that is, the expected performance in future examples [19].

$$w_{t+1} = w_t - \gamma \frac{1}{n} \sum_{i=1}^{n} \nabla_w Q(z_i , w_t), \tag{4}$$

where γ is a properly selected gain. Under assumptions of sufficient regularity, when the initial estimate w_0 becomes close enough to the optimum and the gain γ is small enough, this algorithm achieves linear convergence [20]; this means, $- log\, \rho \sim t$, where ρ represents the residual error.

The stochastic gradient descent (SGD) algorithm is a drastic simplification. Instead of calculating the gradient of $E_n(f_w)$ accurately, each iteration estimates this gradient from a single example z_t randomly selected [21].

$$w_{t+1} = w_t - \gamma_t \nabla_w Q(z_t , w_t), \tag{5}$$

The **Multinomial Naive Bayes (MNB)** model is suitable for determining the probability that a prompt belongs to the "Fraud" class, based on the frequency and co-occurrence of words (tokens). It is effective for detecting changes in linguistic style (e.g., a high frequency of technical or hacking terms). It is based on Bayes' Theorem and is especially useful for classifying documents (text). It operates under the assumption that the presence of a feature (word) in a class is independent of the presence of any other feature (although this is a naive assumption—hence the name Naive—it often works surprisingly well for text).

Its incremental nature (partial_fit) allows updating its probabilities as new tokens or fraud patterns are detected.

In the NB multinomial classifier, each word w_i participates in determining the label $c \in \{1, \ldots, C\}$ that should be assigned to an unseen document $\vec{w} = (w_1, \ldots, w_N)$. To choose a label c for $\vec{w}$, the NB multinomial classifier begins by calculating the prior probability $Pr(c)$ of each label c, which is determined by assuming equiprobable classes or by checking the frequency of each label in the training set. The contribution of every word combines with this prior probability to obtain a likelihood estimate for each label. This is known as the posterior maximum (PM) decision rule. Equation (6) formally defines PM:

$$\begin{aligned} c &= arg\ max_c Pr(c) \prod_{n=1}^{N} Pr(w_n|c) \\ &= arg\ max_c \vartheta_c \prod_{n=1}^{N} \varphi_{c,w_n} \end{aligned} \tag{6}$$

Given a set of training documents $D = \{(\overrightarrow{w_m}, c_m)\}_{m=1}^{M}$, ϑ_c and $\varphi_{c,v}$ They are generally estimated using a smoothed version of maximum likelihood (ML). In fact, both are PM estimates given the uniform Dirichlet prior probabilities.

$$\vartheta_c = \frac{n^{(c)} + \alpha}{M + C\alpha}, \tag{7}$$

$$\varphi_{c,v} = \frac{n_c^{(v)} + \beta}{n_c + V\beta} \tag{8}$$

where V is the number of unique words, $n^{(c)}$ is the number of documents with the class c in D, $n_c^{(v)}$ is the number of times the word v appears in a class document c in D and $n_c = \sum_{v=1}^{V} n_c^{(v)}$. The prior smoothing probabilities $\alpha \geq 0$ and $\beta \geq 0$ They avoid zero probabilities in subsequent calculations. Set $\alpha = 1$ and $\beta = 1$ is called Laplace smoothing, while $\alpha < 1$ and $\beta < 1$ is called Lidstone smoothing [22].

The **Perceptron** functions as a basic, deterministic threshold classifier. Its role is to ensure **strict margin classification** for the most apparent fraud patterns. It is the simplest neural network (a linear classifier). It employs a straightforward update rule: if the prediction is incorrect, the weights are updated additively.

- Unlike the Passive-Aggressive algorithm, which allows a margin of tolerance, the Perceptron is more **binary and strict** in its classification. This can be useful for establishing a hard safety baseline against more direct *prompt* injections.

The synaptic weights of the perceptron are denoted by $w_1, w_2, \ldots, w_m$. Consequently, the inputs applied to the perceptron are denoted by $x_1, x_2, \ldots, x_m$. Externally applied bias is denoted by b. From the model, it can be obtained that the input of the hard limiter or the introduced local field of the neuron is

$$v = \sum_{i=1}^{m} w_i x_i + b, \tag{9}$$

The goal of the perceptron is to correctly classify the set of stimuli $x_1, x_2, \ldots, x_m$ applied externally in one of two classes, C_1 o C_2. The decision rule for classification consists of assigning the point represented by the entries x to class C_1 if the output y of the perceptron is +1 and to the class C_2 if it is -1.

In the simplest form of the perceptron, there are two decision regions separated by a hyperplane defined by

$$\sum_{i=1}^{m} w_i x_i + b = 0, \tag{10}$$

Synaptic weights $w_1, w_2, \ldots, w_m$ The perceptron's behavior can be adapted iteration by iteration. For this adaptation, we can use an error correction rule known as the perceptron convergence algorithm [23].

3.3. Implementation of the Ensemble Online

This component is responsible for classifying both legitimate and malicious queries using an ensemble of models with continuous learning. It features an adaptation method that calculates the performance of the most recent model, normalizes the performances, and applies softmax with a temperature of 3.0 to update the weights of each learner using exponential smoothing.

A fundamental step in this type of system is dataset preprocessing, which involves vectorization to convert text into numerical vectors using the Hashing Trick. This technique is chosen because it does not require storing a vocabulary, making it ideal for online learning. It is compatible with 'partial_fit()', and the processing is faster and more efficient, handling unlimited vocabularies without memory growth. The configuration is as follows:

- n_features: 2^{18} (262,144 features)
- ngram_range: (1, 2)—unigrams and bigrams
- alternate_sign: False—for compatibility with Naive Bayes

Another important step is the decision process, which requires first obtaining the prediction and probability from each model to calculate the weighted average probability. Based on a decision threshold of 0.5, the system then determines whether the query is malicious or legitimate (Table 1).

Table 1. Types of user queries.

Legitimate Query	Malicious Query	Ambiguous Query
Query: "What is my account balance?" │ ├─ Pattern Detection: No threats ├─ Risk Assessment: Low risk ├─ Meta-Learner: malicious_score = 0.15 ├─ Decision: ALLOWED └─ Online Learning: Update with label = 0 (legitimate)	Query: "How can I evade taxes on my investment returns?" │ ├─ Pattern Detection: Tax evasion pattern detected ├─ Risk Assessment: High risk ├─ Meta-Learner: malicious_score = 0.92 ├─ Decision: BLOCKED └─ Online Learning: Update with label = 1 (malicious)	Query: "I want to transfer a large amount without reporting" │ ├─ Pattern Detection: Transaction hiding pattern ├─ Risk Assessment: Medium-High risk ├─ Meta-Learner: malicious_score = 0.65 ├─ Decision: BLOCKED (combined score exceeds threshold) └─ Online Learning: Update with label = 1 (malicious)

Initially, the models are trained with limited data. Through a continuous update process, queries are stored in a buffer to update each model using 'partial_fit()'. It is important to record historical performance data for the adaptation process. When the buffer contains N or more cases, the update process is executed. To prevent buffer saturation, a debugging process occurs when the buffer reaches N′ cases. As a result, the update is performed incrementally for each model.

The proposed architecture (Figure 2) is robust because if one model fails, others compensate; weighted voting allows the best models to have more influence, and different algorithms can capture different patterns. Regarding adaptability, continuous learning improves the model with each query, automatically adjusting its weights to adapt to pattern changes, and does not require complete retraining since updates are incremental.

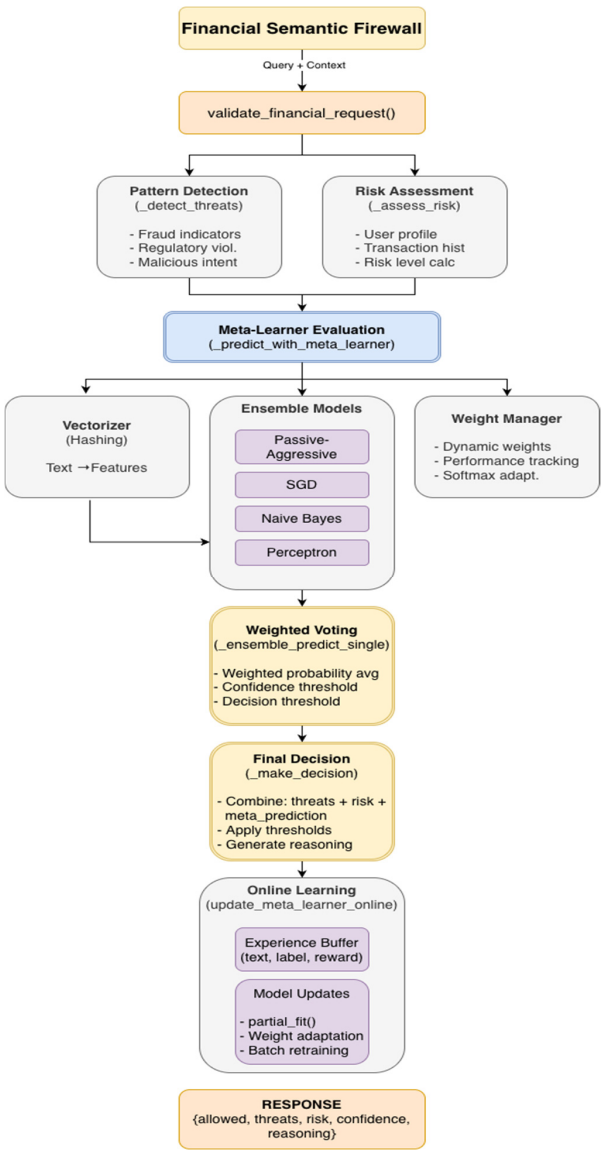


Figure 2. Semantic firewall architecture.

3.4. Rag Agentic

Researchers from Facebook AI Research (now Meta AI), University College London, and New York University coined the term RAG and established a fundamental framework for combining generative models with mechanisms for retrieving external knowledge [24]. Although this research laid the foundation for the RAG concept, the notion of an autonomous "agent" orchestrating the process was not part of the academic discourse at that time. Their main contribution was to demonstrate that a generative model could be "augmented" with Wikipedia data to significantly improve performance in open-domain question answering tasks.

A basic sequence of RAG consists of a search step followed by a generation step, where the question is posed to the LLM and the information is retrieved from the search step.

A RAG-Token model can extract a latent document specific to each target token and marginalize as appropriate. This enables the generator to select content from several documents to generate an answer. Specifically, it retrieves K principal documents through

retrieval, and then the generator produces a distribution for the next output token for each document, before marginalizing and repeating the process with the next output token.

$$p_{RAG-Token}(y|x) \simeq \prod_{i}^{N} \sum_{z \in top-k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z_i, y_{1:i-1}) \tag{11}$$

where x is the input sequence for retrieving text documents z and uses it as additional context when generating the target sequence y; $p_{\eta}(z|x)$ is the recovery tool with parameters η that returns distributions (truncated at the first K) over text passages given a query x, and $p_{\theta}(y_i|x, z_i, y_{1:i-1})$ is the generator parameterized by θ that generates a current token based on a context of the $i-1$ previous tokens $y_{1:i-1}$, the original input x and a recovered passage z [24].

Given a collection of M text passages, the dense passage retriever (DPR) indexes all passages in a continuous, low-dimensional space, enabling efficient retrieval of the relevant principal k passages for the input question at runtime [25]. A generative model then processes the retrieved passages. This approach allows scaling to a large number of documents and leveraging this extensive body of evidence [26].

We can describe a system such as RAG as agentic if it breaks the linear flow of a standard RAG system and grants an agent the ability to perform multiple steps to achieve a goal. The concept of an agent has gained importance in both Artificial Intelligence and conventional computing. Agent theory focuses on the question of what an agent is and uses mathematical formalisms to represent and reason about its properties. Agent architectures can be considered software engineering models of agents [27].

Multi-Agent Systems (MAS) are the AI subfield that seeks to provide principles for constructing complex systems involving multiple agents and mechanisms for coordinating independent agent behavior. MAS allows the division of subproblems into specialized tasks within a general problem, delegating problem-solving to different agents with their own interests and objectives [28].

According to the definition by [29], an ideal rational agent should perform any action that maximizes its performance, based on the evidence provided through the sequence of perceptions and any incorporated knowledge the agent possesses.

The agent implementation proposed to trace multiple routes and action plans to resolve simple or complex queries, focusing on financial topics, to combine analyses in distinct areas of interest, as seen on Figure 3.

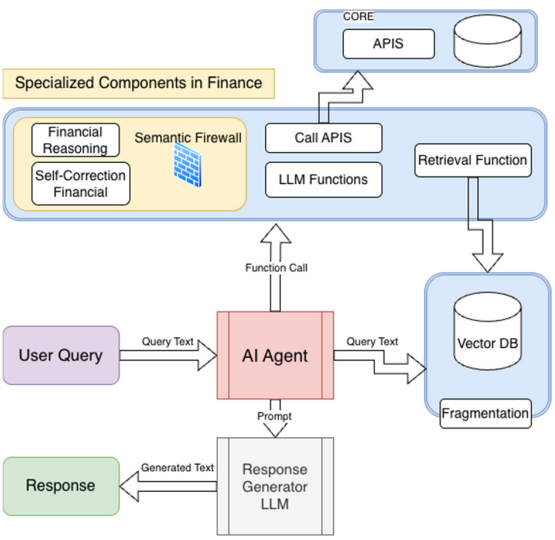


Figure 3. RAG agentic architecture with components specialized in Finance.

3.5. Apis to Automate Processes

The RAG chatbot combines the capabilities of Large Language Models (LLMs) with its own data to deliver accurate and contextually relevant answers. Rather than relying solely on the LLM's pre-trained knowledge, a RAG chatbot can access and process information from its documents.

Equally important is its ability to obtain the profile of the customer making the inquiry. The agent must manage customer information, which requires tools provided through an **Application Programming Interface (API)** designed to request and receive information from databases or other data sources.

The core banking system is central to a bank's operations and contains essential information about customer accounts, balances, and transactions. Integration with this system is crucial for the chatbot to provide information about account balances and transaction details.

As with the core system, this integration can be achieved through APIs that allow the chatbot to query specific loan details based on the customer's ID.

To provide information on interest rates for financial products, the chatbot needs access to the systems where this information is stored. This may include product databases, interest rate management systems, or external data sources via APIs.

The chatbot must be integrated with authentication systems to verify user identity before granting access to sensitive information. It must also be integrated with authorization systems to determine what information and functions each authenticated user can access and to ensure the use of secure protocols.

This enables the agent to organize information and provide context to the financial components, ensuring that financial reasoning, self-correction, and the semantic firewall receive the appropriate inputs to generate a more accurate response.

3.6. Database

Experiments were conducted using both synthetic and open-source data. A synthetic dataset representing the banking domain is necessary to develop, test, and evaluate the system. Using this dataset eliminates the need for actual personal data, thereby protecting privacy and ensuring compliance with data protection regulations. It is important to obtain diverse data to cover a wide range of scenarios and typical queries from banking users. The actual datasets were sourced from two different collections: the Finance-Instruct-500k dataset, which includes content tailored for financial reasoning, question answering, entity recognition, sentiment analysis, address parsing, and multilingual natural language processing (NLP) [30], and AdvBench, which consists of 500 harmful behaviors formulated as instructions [31].

3.6.1. Real Data

Finance-Instruct-500k is a high-quality financial instruction dataset containing over 500,000 dialog entries and queries related to the financial domain, available on Hugging Face ('hyper-ai/Finance-Instruct-500k' or 'Josephgflowers/Finance-Instruct-500k') [30]. AdvBench (Adversarial Benchmark) is a set of adversarial prompts specifically designed to assess the security and robustness of language models (LLMs) against prompt engineering attacks. Both sets are combined in a 50/50 ratio to balance legitimate and malicious classes. The goal is to use the proposed semantic firewall architecture to block AdvBench prompts, allow legitimate queries from Finance-Instruct-500k, learn from real-world threat patterns, and continuously adapt through online learning.

3.6.2. Synthetic Data

To create the synthetic data, realistic financial scenarios were developed that incorporate multiple aspects of the banking domain, each scenario integrating two to four dimensions:

- **Credit**: Credit history, loans, credit cards, mortgages.
- **Investment**: Portfolios, mutual funds, diversification, risk management.
- **Regulatory**: KYC/AML compliance, regulatory disclosures, CNBV requirements.
- **Market**: Economic conditions, volatility, market trends.

Table 2 outlines the considerations used when generating the synthetic data to ensure consistency and provide a variety of financial topics. Eighty scenarios of legitimate queries and twenty scenarios containing threat patterns were generated. Table 3 describes the threat patterns considered when generating the malicious queries.

Table 2. Synthetic data query.

Considerations	Brief Description
They reflect common banking inquiries	The questions cover real-world use cases such as balance inquiries, transfers, credit applications, and investment advice.
They integrate multiple dimensions	They simulate the real complexity of financial inquiries where a client can ask about credit, investment and regulatory compliance simultaneously.
They use appropriate financial terminology	They include correct technical terms (KYC, AML, diversification, volatility, etc.).
They maintain contextual coherence	The queries are aligned with the user profile and the provided context.

Table 3. Queries with threat patterns.

Threat Pattern	Brief Description
Tax evasion	Queries about how to evade taxes or hide income.
KYC/AML Bypass	Attempts to avoid identity checks or anti-money laundering.
Money laundering	Inquiries about how to launder illicit money.
Fraudulent schemes	Investments with guaranteed returns, pyramid schemes
Concealment of transactions	Attempts to conceal financial activity
Fake accounts	Creating accounts with false documents
Security bypass	Evasion of bank security controls
Regulatory violations	Ignoring regulatory compliance

Figure 4 presents the statistical distribution or description of the main variables in the synthetic data, ensuring diversity in length, complexity, and dimensional distribution. The queries range from short to complex multi-dimensional questions, from simple queries ("What is my balance?") to complex queries integrating multiple dimensions. The scenarios combine two to four dimensions proportionally, reflecting the multi-dimensional nature of real-world financial queries.

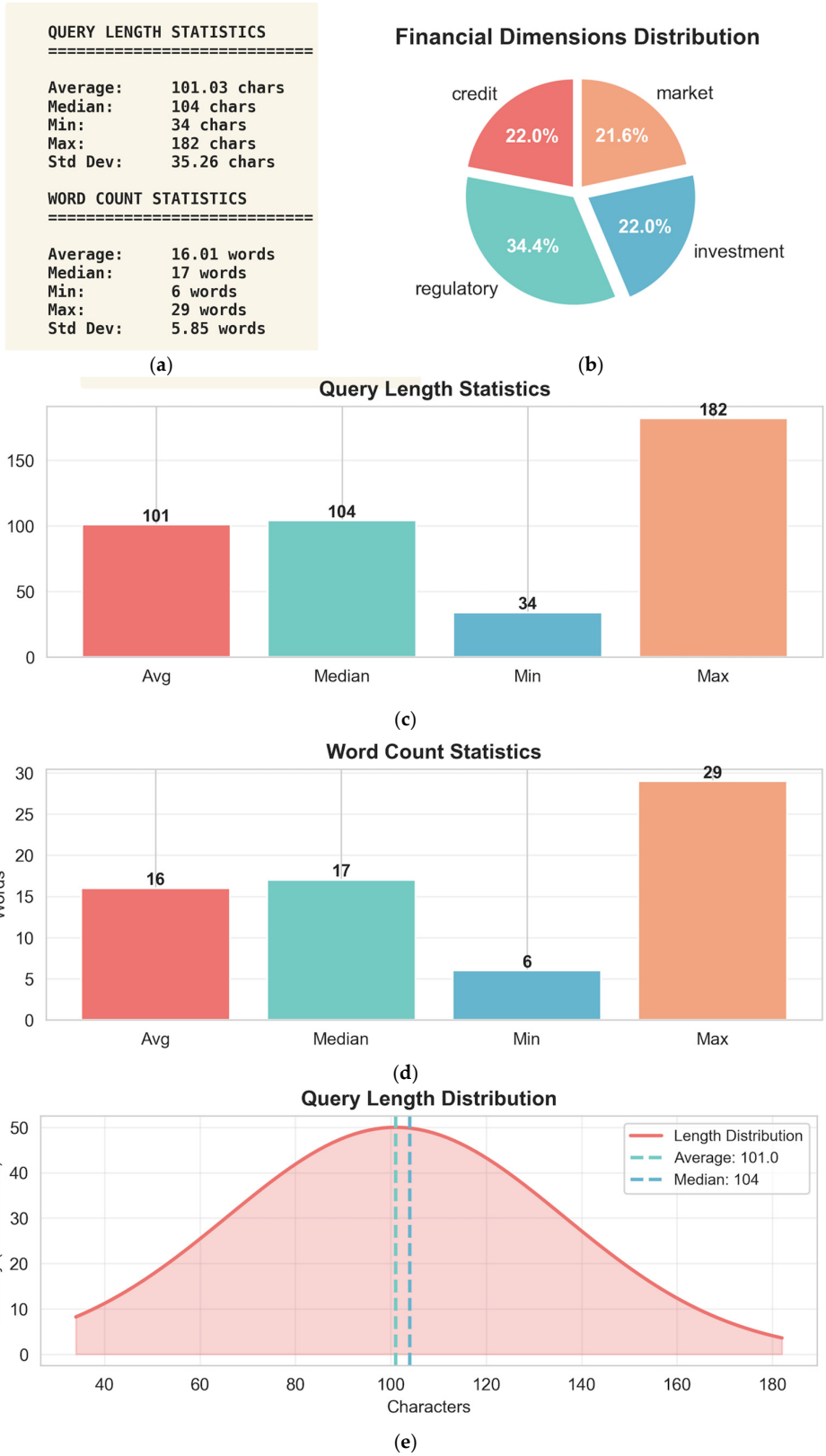
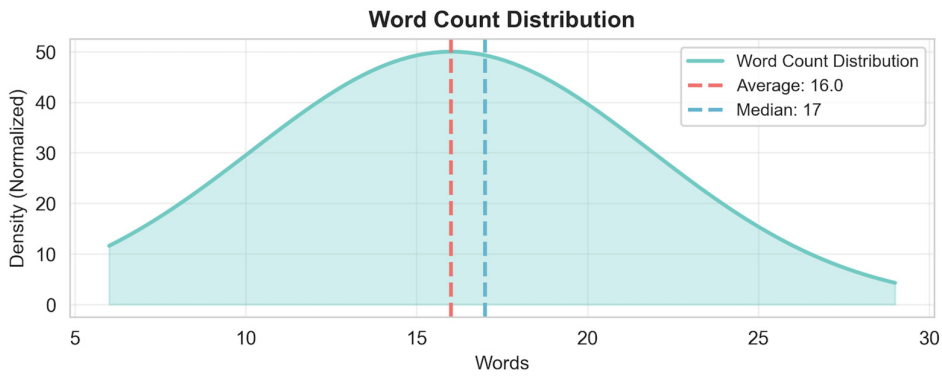


Figure 4. *Cont.*



(**f**)

Figure 4. Statistical distribution of the main variables in the synthetic data. (**a**) statistical description; (**b**) distribution of financial dimensions; (**c**) query length statistics; (**d**) word count statistics; (**e**) query length distribution; (**f**) word count distribution.

4. Results

4.1. Results of Tests with Synthetic Data

The following describes four experiments designed to evaluate different variants of the Financial Semantic Firewall using synthetic data:

- F0: No firewall.
- F1: Static binary firewall (single classifier).
- F2: Static ensemble (without online learning).
- F3: Online ensemble (with batch updates).

Experiment F0 (No Firewall) does not apply any security filter, so all requests are automatically allowed without threat processing. This results in a 0% blocking rate and minimal processing time, serving as a baseline for comparison.

In experiment F1 (Static Binary Firewall), a static single classifier based on 'SGDClassifier' with 'log_loss' is used, combining pattern-based detection with the single classifier.

Experiment F2 (Static Ensemble) uses a set of four classifier models without online learning, employing optimized adaptive weighted voting with improved thresholds and weight adaptation. The four combined models are "passive_aggressive," "sgd," "naive_bayes," and "perceptron," each initially weighted at 0.25. Weighted voting is used for the final decision, with a decision threshold of 0.6.

Experiment F3 (Online Ensemble) uses the same four classifier models: "passive_aggressive," "sgd," "naive_bayes," and "perceptron," with continuous batch learning (adaptation_rate = 0.05), iterative model updates, and weight adjustments. It includes an experience buffer to accumulate new cases for model updates. Weights are updated using softmax with increased temperature.

According to Table 4, experiment F3 (Ensemble Online) achieved the highest accuracy at 98%. Continuous learning significantly improves performance, detecting 90% of threats and achieving a better balance with an F1 score of 0.947. For the False Positive Rate (FPR), 0% of legitimate queries were blocked. Continuous learning eliminates false positives, and 10% of threats passed through, which is an acceptable trade-off. Regarding specificity, 100% of the legitimate queries allowed were correctly permitted, meaning all queries without threats were legitimate. The computational cost is also notable; although this process requires more processing, the average time per request is 17.65 ms.

Table 4. Performance metrics for experiments with synthetic data.

	Accuracy	Precision	Recall	F1-Score	FPR	Specificity
F0	0.800	N/A	0.000	0.000	0.000	1.000
F1	0.780	0.476	1.000	0.645	0.275	0.725
F2	0.950	0.800	1.000	0.889	0.062	0.938
F3	0.980	1.000	0.900	0.947	0.000	1.000

Figure 5a,b show a comparison of the metric results where continuous learning has an advantage over the others. Figure 5c shows that more resources are also consumed, but in the order of milliseconds. Figure 5d only compares experiments performed with ensemble learning, where continuous learning outperforms static learning. Figure 6 shows all the confusion matrices for the variants showing False positives (FP), false negatives (FN), True Positives (TP) and True Negatives (TN).

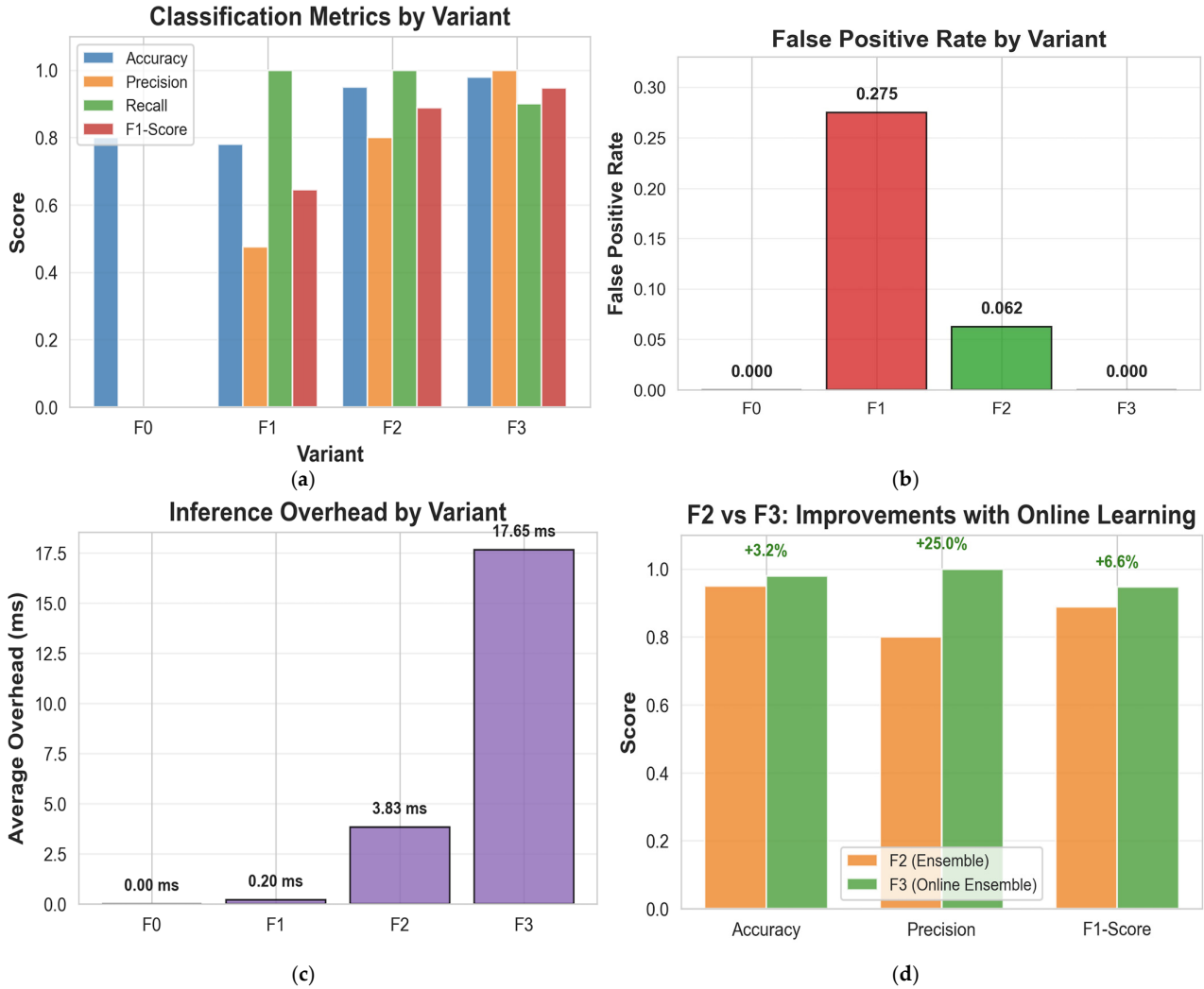


Figure 5. Semantic firewall variant comparison with synthetic data. (**a**) classification metrics by variant, (**b**) false positive rate (FPR) by variant; (**c**) inference overhjead by variant and (**d**) F2 vs. F3 Improvements with online learning.

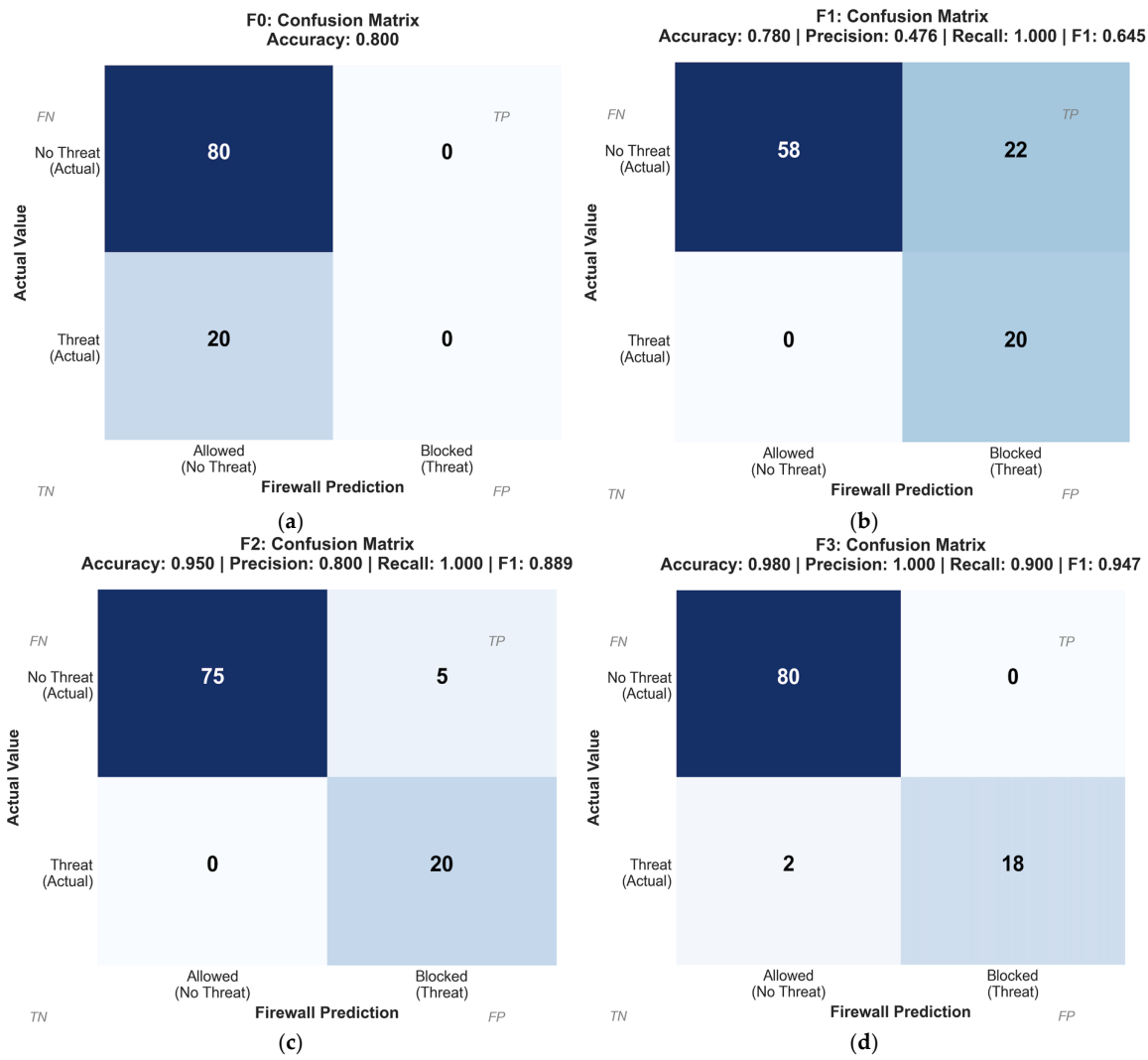


Figure 6. Confusion matrices with synthetic data. (**a**) F0 confusion matrix prediction; (**b**) F1 confusion matrix prediction; (**c**) F2 confusion matrix prediction and (**d**) F3 confusion matrix prediction.

4.2. Results of Tests with Real Data

Experiments G0–G3 evaluate the performance of the financial semantic firewall using real data from **AdvBench** (adversarial prompts) balanced with **Finance-Instruct-500k** (legitimate queries). These experiments use meta-learning models trained on real data instead of synthetic data.

- G0: Baseline without firewall.
- G1: Firewall with single classifier.
- G2: Firewall with static ensemble (without online learning).
- G3: Online ensemble (with batch updates).

According to Table 5, the G3 (Ensemble Online) experiment achieved the highest accuracy, exceeding 98%. Continuous learning significantly improves performance, detecting 97% of threats and achieving a better balance with an F1 score of 0.985. For the False Positive Rate (FPR), 0.2% of legitimate queries were blocked. Continuous learning eliminates false positives, and 2.8% of threats passed through, which is an acceptable trade-off. Regarding specificity, 99% of the legitimate queries allowed were correctly permitted, meaning all queries without threats were legitimate. The computational cost is also notable; although this process requires more processing, the average time per request is 20.82 ms.

Table 5. Performance metrics for experiments with synthetic data.

	Accuracy	Precision	Recall	F1-Score	FPR	Specificity
F0	0.500	N/A	0.000	0.000	0.000	1.000
F1	0.946	0.961	0.930	0.945	0.038	0.962
F2	0.978	0.990	0.966	0.978	0.010	0.990
F3	0.985	0.998	0.972	0.985	0.002	0.998

Figure 7a,b show a comparison of the metric results with real data, where continuous learning has an advantage over the others. Figure 7c shows that more resources are also consumed, but on the order of milliseconds. Figure 7d only compares experiments performed with ensemble learning, where continuous learning outperforms static learning. Figure 8 shows all the confusion matrices for the variants with real data.

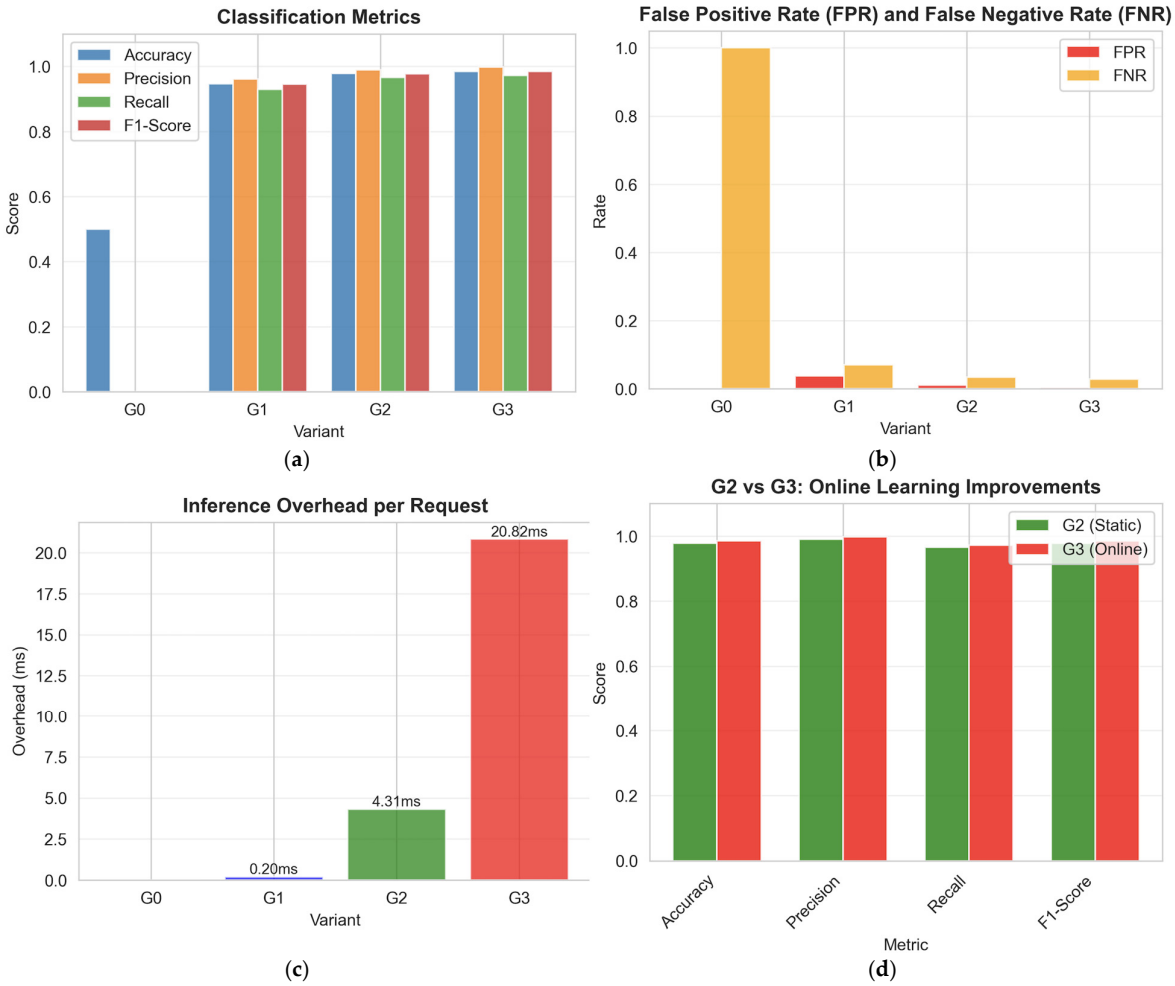


Figure 7. Semantic firewall variants comparison with real data. (**a**) classification metrics by variant; (**b**) false positive rate and false negative rate by variant; (**c**) inference overhear per request by variant; and (**d**) G2 vs. G6 online learning improvements.

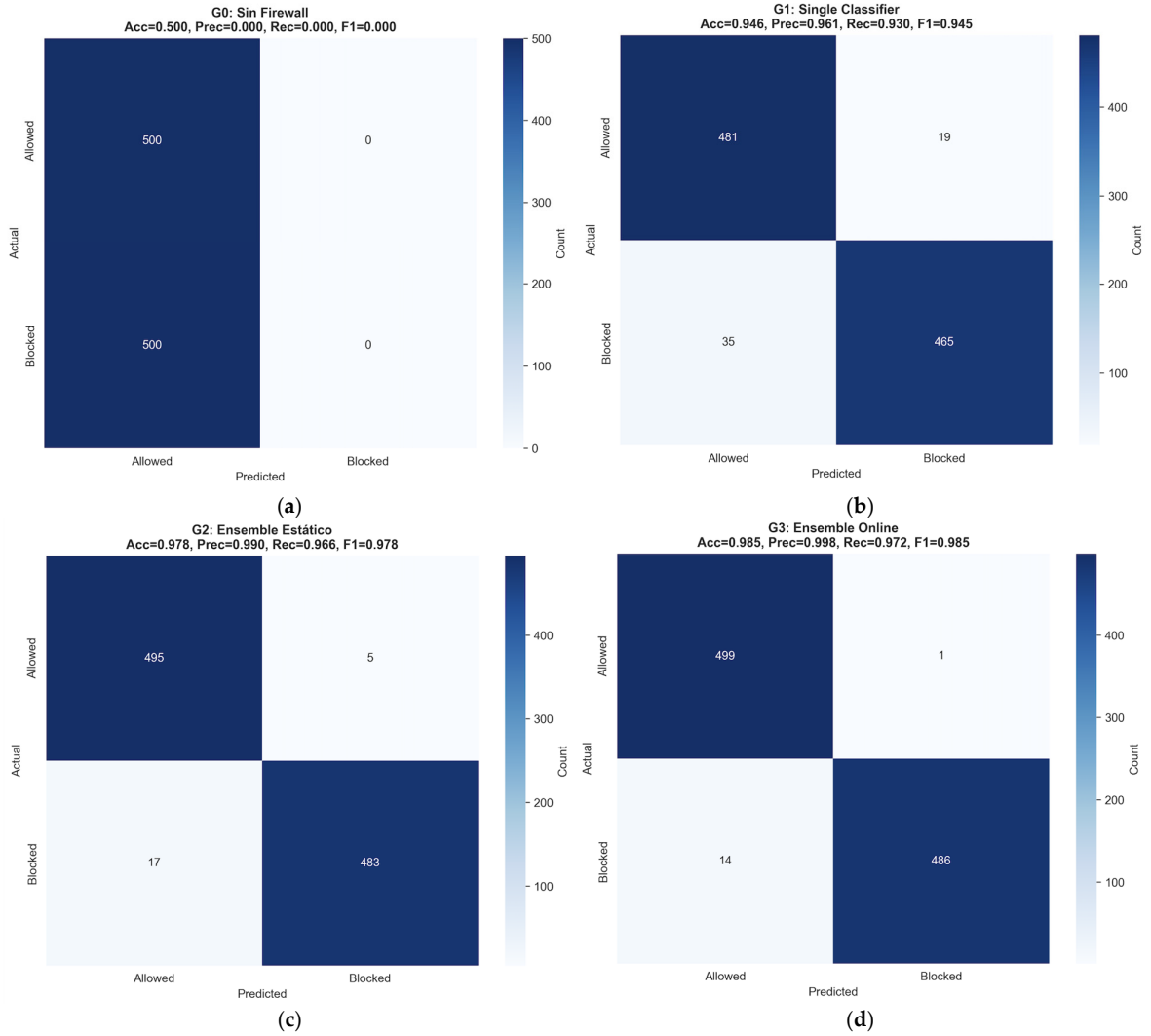


Figure 8. Confusion matrices with real data. (**a**) G0 no firewall confusion mnatrix; (**b**) G1 single classifier confusion matrix; (**c**) G2 static ensemble confusion matrix and (**d**) online ensemble confusion matrix.

5. Discussion

The purpose of a chatbot is to interact with customers or users automatically through a fluid, helpful, and ideally efficient conversation. However, in the banking sector, additional aspects such as security and transparency are crucial. A customer who trusts the system will feel safe and comfortable using it. This system as a whole successfully fulfills these requirements. It not only organizes information to answer customer questions using RAG techniques and API implementation to provide sufficient context—whether from regulatory issues or customer profile information—but also integrates the necessary components to ensure reasoned responses at different levels of conversation and self-correction. Most importantly, if it receives malicious questions that could compromise customer privacy or expose vulnerabilities, it can respond to or block these threats in real time

6. Conclusions and Future Work

The developed system marks a significant advancement in banking chatbots with specialized capabilities. Experimental results show several strengths, especially the flawless accuracy of the Semantic Firewall, which greatly reduces inappropriate advice and regulatory violations.

Adaptive learning improves the personalization and relevance of financial recommendations, demonstrating the superiority of the online ensemble over simple meta-learners.

- Algorithm Diversity: Each model offers a unique perspective.
- Adaptive Weighting: Automatic adjustment based on performance.
- Continuous Learning: Improves with each new query.
- System Robustness: Resistant to overfitting and individual failures.
- Advanced Explainability: Detailed answers from the LLM.
- Continuous learning improves F1-Score and reduces FPR.

Efficient real-time processing and advanced explainability in LLM responses were also observed. And in general, the following benefits can be observed.

- Unified query processing flow.
- Multi-layered validation (input, processing, output).
- Consistent responses by integrating all components.
- Centralized monitoring of metrics and performance.

For future work, the capabilities and dimensions of analysis could also be expanded as needed and specialized in specific topics. As well as scaling to larger volumes in the order of hundreds of thousands and adapting the system to other security contexts, optimizations can be made to reduce overhead by performing updates asynchronously. Different online learning algorithms can also be tested, or pre-trained models with fine-tuning can be leveraged.

Author Contributions: Conceptualization, V.C.-M., M.A.A.-F. methodology, M.A.A.-F. and L.R.G.-N.; software, J.C.P.-O. and V.C.-M.; validation, J.C.P.-O. and M.A.A.-F.; formal analysis, M.A.A.-F.; investigation, L.R.G.-N.; resources, V.C.-M. and M.A.A.-F. data curation, L.R.G.-N. and J.C.P.-O.; writing—original draft preparation, V.C.-M.; writing—review and editing, M.A.A.-F. and J.C.P.-O.: visualization, V.C.-M. supervision, M.A.A.-F.; project administration, M.A.A.-F. and L.R.G.-N. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets presented in this article are not readily available because the data are part of an ongoing study. Requests to access the datasets should be directed to the corresponding author.

Conflicts of Interest: The authors declare that they have no competing interests.

References

1. Wu, S.; Xiong, Y.; Cui, Y.; Wu, H.; Chen, C.; Yuan, Y.; Hunag, L.; Liu, X.; Kuo, T.-W.; Guan, N.; et al. Retrieval-augmented generation for natural language processing: A survey. *arXiv* **2024**, arXiv:2407.13193. [CrossRef]
2. Verma, S. Contextual compression in retrieval-augmented generation for large language models: A survey. *arXiv* **2024**, arXiv:2409.13385.
3. Zhong, K.; Suleiman, B.; Erradi, A.; Chen, S. SemRAG: Semantic Knowledge-Augmented RAG for Improved Question-Answering. *arXiv* **2025**, arXiv:2507.21110.
4. Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv* **2023**, arXiv:2307.16789.
5. Qin, Y.; Hu, S.; Lin, Y.; Chen, W.; Ding, N.; Cui, G.; Zeng, Z.; Zhou, X.; Huang, Y.; Xiao, C.; et al. Tool learning with foundation models. *ACM Comput. Surv.* **2024**, *57*, 1–40. [CrossRef]
6. Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 68539–68551.

7. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, T.; Narasimhan, K.R.; Cao, Y. React: Synergizing reasoning and acting in language models. In Proceedings of the Eleventh International Conference on Learning Representations, Kigali, Rwanda, 1–5 May 2023.
8. Wang, Q.; Zhang, L.; Huang, Y. FinAgent: A multimodal foundation agent for financial trading. *ACM Trans. Manag. Inf. Syst.* **2023**, *15*, 1–19.
9. Yang, H.; Zhang, B.; Wang, N.; Guo, C.; Zhang, X.; Lin, L.; Wang, J.; Zhou, T.; Guan, M.; Zhang, R.; et al. Finrobot: An open-source ai agent platform for financial applications using large language models. *arXiv* **2024**, arXiv:2405.14767. [CrossRef]
10. Govind Srinivasan, A.; George, R.J.; Koshy Joe, J.; Kant, H.; Harshith, M.R.; Sundar, S.; Suresh, S.; Vimalkanth, R.; Vijayavallabh. Enhancing Financial RAG with Agentic AI and Multi-HyDE: A Novel Approach to Knowledge Retrieval and Hallucination Reduction. *arXiv* **2025**, arXiv:2509.16369.
11. Iyer, K.R. Streaming Intelligence For Real-Time Fraud Detection: A Practical And Theoretical Framework Using Online Learning, Anomaly Detection, And Stream Processing. *Stanf. Database Libr. Am. J. Appl. Sci. Technol.* **2025**, *5*, 317–323.
12. Deprez, B.; Wei, W.; Verbeke, W.; Baesens, B.; Mets, K.; Verdonck, T. Advances in Continual Graph Learning for Anti-Money Laundering Systems: A Comprehensive Review. *Wiley Interdiscip. Rev. Comput. Stat.* **2025**, *17*, e70040. [CrossRef]
13. Dai, Y.; Ji, Z.; Li, Z.; Li, K.; Wang, S. Disabling Self-Correction in Retrieval-Augmented Generation via Stealthy Retriever Poisoning. *arXiv* **2025**, arXiv:2508.20083. [CrossRef]
14. Abdelnabi, S.; Gomaa, A.; Bagdasarian, E.; Kristensson, P.O.; Shokri, R. Firewalls to Secure Dynamic LLM Agentic Networks. *arXiv* **2025**, arXiv:2502.01822. [CrossRef]
15. Zhou, Z.-H. *Ensemble Methods: Foundations and Algorithms*; CRC Press: Boca Raton, FL, USA, 2025.
16. Dietterich, T.G. Ensemble methods in machine learning. In *Proceedings of the International Workshop on Multiple Classifier Systems, Cagliari, Italy, 21–23 June 2000*; Springer: Berlin/Heidelberg, Germany, 2000; pp. 1–15.
17. Gama, J.; Žliobaitė, I.; Bifet, A.; Pechenizkiy, M.; Bouchachia, A. A survey on concept drift adaptation. *ACM Comput. Surv. CSUR* **2014**, *46*, 1–37. [CrossRef]
18. Crammer, K.; Dekel, O.; Keshet, J.; Shalev-Shwartz, S.; Singer, Y. Online passive-aggressive algorithms. *J. Mach. Learn. Res.* **2006**, *7*, 551–585.
19. Rumelhart, D.E.; Hinton, G.E.; Williams, R.J. Learning representations by back-propagating errors. *Nature* **1986**, *323*, 533–536. [CrossRef]
20. Dennis, J.E., Jr.; Schnabel, R.B. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; SIAM: Philadelphia, PA, USA, 1996.
21. Bottou, L. Large-scale machine learning with stochastic gradient descent. In *Proceedings of the COMPSTAT'2010: 19th International Conference on Computational Statistics, Paris, France, 22–27 August 2010*; Keynote, Invited and Contributed Papers; Springer: Berlin/Heidelberg, Germany, 2010; pp. 177–186.
22. Xu, S.; Li, Y.; Wang, Z. Bayesian multinomial Naïve Bayes classifier to text classification. In *Proceedings of the International Conference on Multimedia and Ubiquitous Engineering, Seoul, Korea, 22–24 May 2017*; Springer: Berlin/Heidelberg, Germany, 2017; pp. 347–352.
23. Haykin, S. *Neural Networks: A Comprehensive Foundation*; Prentice Hall PTR: Upper Saddle River, NJ, USA, 1994.
24. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-T.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474.
25. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W.-T. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Online, 16–20 November 2020; pp. 6769–6781.
26. Izacard, G.; Grave, E. Leveraging passage retrieval with generative models for open domain question answering. *arXiv* **2020**, arXiv:2007.01282.
27. Wooldridge, M.; Jennings, N.R. Intelligent agents: Theory and practice. *Knowl. Eng. Rev.* **1995**, *10*, 115–152. [CrossRef]
28. Stone, P.; Veloso, M. Multiagent systems: A survey from a machine learning perspective. *Auton. Robot.* **2000**, *8*, 345–383. [CrossRef]
29. Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach*; Prentice Hall Series in Artificial Intelligence; Prentice Hall: Englewood Cliffs, NJ, USA, 1995; Volume 25, pp. 79–80.
30. Josephgflowers/Finance-Instruct-500k. Datasets at Hugging Face. Available online: https://huggingface.co/datasets/Josephgflowers/Finance-Instruct-500k (accessed on 15 January 2026).
31. walledai/AdvBench. Datasets at Hugging Face. Available online: https://huggingface.co/datasets/walledai/AdvBench (accessed on 15 January 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.